



MANUAL DE INTEGRACIÓN

API de Skala Fintech

Guía completa para que una plataforma externa se integre con Skala: **radicar créditos**, consultar su estado y datos, cambiar de estado, gestionar **devoluciones con tareas**, agregar comentarios y recibir eventos por **webhooks**.

Versión 1.2

Actualizado 2026-07-15

REST · JSON · HTTPS

CONTENIDO

- | | |
|----------------------------------|-------------------------------------|
| 01 Introducción | 07 Ciclo de vida del crédito |
| 02 Conceptos clave | 08 Webhooks y polling |
| 03 Primeros pasos | 09 Manejo de errores |
| 04 Autenticación y scopes | 10 Buenas prácticas |
| 05 Convenciones | 11 Catálogos de referencia |
| 06 Endpoints | 12 Preguntas frecuentes |

1 Introducción

Skala es una plataforma de gestión de crédito de libranza. Su API permite que **plataformas de terceros** (originadores, aliados, marketplaces) operen contra Skala de forma programática.

● Entrada · tu plataforma → Skala

Crear créditos, consultarlos, cambiar su estado, devolverlos con tareas de subsanación y comentar.

● Salida · Skala → tu plataforma

Recibir notificaciones de eventos (webhooks) cuando algo cambia en tus créditos.

La API es **REST sobre HTTPS**, con **JSON** en request y response, y autenticación por **API key**.

2 Conceptos clave

Concepto	Descripción
Crédito	Solicitud de libranza. Tiene un <code>solicitud_number</code> (entero único) que es su identificador de negocio; lo recibes al radicar.
Estado	Etapas del crédito en el flujo (ej. <i>RADICADO / PTE VALIDACIÓN, EN ESTUDIO - ANALISTA, DESEMBOLSADO</i>).
Radicación	Creación de un crédito. Nace en el estado inicial del flujo.
Devolución	Cambio a un estado que habilita tareas (<code>enable_tasks</code>), para pedirle al originador que subsane algo. Lleva una lista de tareas.
Comentario	Nota en el hilo del crédito. Puede ser de un usuario/plataforma o de sistema (generada automáticamente).
Gestor	Asesor de Skala dueño del crédito. Opcional al radicar por API.
Entidad	Entidad aliada que financia (ej. <i>CREDIALIANZA</i>).
Pagaduría	Entidad que paga la nómina/pensión del cliente y de donde se descuenta la cuota.
Línea de crédito	Producto (ej. <i>LIBRE INVERSION, COMPRA DE CARTERA</i>).

3 Primeros pasos

Base URL

```
https://yfosumpmtmcomfpbspaz.supabase.co/functions/v1/api
```

- **Formato:** JSON (`Content-Type: application/json`).
- **Zona horaria:** las fechas se devuelven en **ISO 8601 (UTC)**.
- **Tu API key:** Skala te emite una llave asignada a tu entidad. Se muestra **una sola vez**; guárdala del lado servidor.

Prueba de conectividad (debe responder **401** si no envías llave, confirmando que el endpoint está vivo):

BASH

```
curl -i 'https://yfosumpmtmcomfbpaz.supabase.co/functions/v1/api/credits/1'
```

4 Autenticación y scopes

Toda petición requiere el header **x-api-key** :

```
x-api-key: sk_skala_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Cada llave tiene **scopes** que limitan lo que puede hacer:

Scope	Permite
credits:create	Radical créditos (POST /credits)
credits:read	Consultar créditos y comentarios (GET ...)
credits:update	Cambiar estado, devolver y comentar (PATCH, POST .../comments)

401 Llave ausente o inválida

403 Llave sin el scope requerido

Seguridad de la llave

- En el servidor de Skala **solo se guarda el hash SHA-256** de tu llave (nunca en texto plano). Si la pierdes, **no es recuperable**: se re-emite.
- Las llaves pueden tener **fecha de expiración**; una llave vencida responde **401**.
- La llave es **secreta**: úsala **solo del lado servidor** (nunca en frontend, app móvil, repos públicos ni logs).
- **Alcance**: tu llave está asignada a tu entidad y opera solo sobre los créditos de esa entidad, con los scopes mínimos que necesites.
- Guárdala en un **gestor de secretos / variable de entorno**, no en el código.
- **Para rotar**: Skala emite una nueva y revoca la anterior. Solicítalo con anticipación para solapar. Reporta de inmediato cualquier sospecha de filtración.

5

Convenciones

- **Identificador del crédito**: el **solicitud_number** (entero). También puedes ubicar por **?cedula=**.
- **Montos**: enteros en pesos colombianos (COP), sin separadores. Ej. **5000000**.
- **Tasa**: número (porcentaje mensual según la entidad). Ej. **1.85**.
- **Idempotencia**: en POST **/credits**, envía el header **Idempotency-Key** (UUID por intento). Si reintentas con la misma llave, la API devuelve el crédito ya creado en vez de duplicarlo.
- **Tamaño del body**: máximo **64 KB**.
- **request_id**: toda respuesta de error incluye un **request_id** para soporte.
- **Saneos**: en la creación, los campos del cliente pasan por una **lista blanca**; las claves no reconocidas se descartan.
- **Aislamiento por entidad**: tu llave está asignada a una entidad/alianza. Solo ves y modificas créditos de **tu entidad**. Créditos de otra entidad responden **404**; crear para otra entidad responde **403**.
- **external_ref (enlace de IDs)**: al radicar puedes enviar tu propio identificador en **external_ref**. Skala lo guarda y te lo devuelve junto con su **solicitud_number**. Reenviarlo no duplica. Consulta por él: GET **/credits?external_ref=TU_ID**.

6 Endpoints

6.1 **POST** /credits

Scope **credits:create**

Radical crédito

Campos del body:

Campo	Tipo	Obl.	Notas
entidad	string	✓	Nombre de la entidad aliada.
monto	number	✓	1 – 2.000.000.000.
plazo	number	✓	Meses, 1 – 240.
tasa	number	✓	0 – 100.
cliente	object	✓	Requiere al menos numeroDocumento o nombres .
montoDesembolso	number	–	Default = monto .
comisionPct	number	–	0 – 100. Si se omite, la comisión estimada queda en 0 (envíala si quieres fijarla).
lineaCredito	string	–	Ver catálogo de líneas.
external_ref	string	–	Tu propio ID del crédito (enlace + dedup).
gestorId	uuid	–	Si se envía, debe ser un gestor existente; si se omite, queda sin asignar.

Objeto **cliente** — campos aceptados

nombres · apellidos · nombreCompleto · tipoDocumento · numeroDocumento (5–15 dígitos) · correo · telefonoCelular · pagaduria · banco · tipoCuenta · numeroCuenta · ciudadResidencia · direccionCompleta · barrio · estadoCivil · sexo · fechaNacimiento · ciudadNacimiento · ciudadExpedicion · fechaExpedicion · tipoPension · mesadaPensional · cuotaUtilizar · observaciones

Cualquier otra clave se ignora.

Ejemplo — curl

BASH

```
curl -X POST 'https://yfosumpmtmcomfpbspaz.supabase.co/functions/v1/api/credits' \
-H 'x-api-key: TU_API_KEY' \
-H 'Content-Type: application/json' \
-H 'Idempotency-Key: 7c1f9a2e-4b3d-4a1e-9f0c-2b6a1d8e5f3a' \
-d '{
  "entidad": "CREDIALIANZA",
  "monto": 5000000,
  "plazo": 60,
  "tasa": 1.85,
  "montoDesembolso": 4800000,
  "comisionPct": 2,
  "lineaCredito": "LIBRE INVERSION",
  "cliente": {
    "nombres": "JUAN CARLOS", "apellidos": "PEREZ GOMEZ",
    "numeroDocumento": "1020304050", "correo": "juan@correo.com",
    "telefonoCelular": "3001234567", "pagaduria": "Colpensiones"
  }
}'
```

Ejemplo — Node.js

JAVASCRIPT

```
const res = await
fetch('https://yfosumpmtmcomfpbspaz.supabase.co/functions/v1/api/credits', {
  method: 'POST',
  headers: {
    'x-api-key': process.env.SKALA_API_KEY,
    'Content-Type': 'application/json',
    'Idempotency-Key': crypto.randomUUID(),
  },
  body: JSON.stringify({
    entidad: 'CREDIALIANZA', monto: 5000000, plazo: 60, tasa: 1.85,
    cliente: { numeroDocumento: '1020304050', nombres: 'JUAN', apellidos: 'PEREZ',
    pagaduria: 'Colpensiones' },
  }),
});
const credito = await res.json(); // { id, solicitud_number, estado }
```

Ejemplo — Python

PYTHON

```
import requests, uuid
r = requests.post(
  'https://yfosumpmtmcomfpbspaz.supabase.co/functions/v1/api/credits',
  headers={'x-api-key': API_KEY, 'Idempotency-Key': str(uuid.uuid4())},
  json={'entidad': 'CREDIALIANZA', 'monto': 5000000, 'plazo': 60, 'tasa': 1.85,
    'cliente': {'numeroDocumento': '1020304050', 'nombres': 'JUAN', 'pagaduria':
    'Colpensiones'}})
print(r.json()) # {'id': ..., 'solicitud_number': 1420, 'estado': 'RADICADO / PTE
VALIDACIÓN'}
```

Respuesta 201

```
{ "id": "uuid", "solicitud_number": 1420, "external_ref": "TU-ID-123 (o null si no lo
enviaste)", "estado": "RADICADO / PTE VALIDACIÓN" }
```

Nota — si el crédito ya existía (mismo **Idempotency-Key** o mismo **external_ref**), la respuesta es **200** (no **201**) con el crédito ya creado — no se duplica.

Regla anti-duplicado (por pagaduría) → 409

No puedes radicar si ya existe un crédito **en trámite** (estado no final) para la **misma cédula o el mismo correo** en la **misma pagaduría**. Puedes radicar otro **solo si es con una pagaduría diferente**, o si el anterior ya llegó a un estado final (DESEMBOLSADO / NEGADO / DESISTIDO).

```
409 { "error": "Ya existe un crédito en trámite para este cliente en la pagaduría \"...\" (estado: ...). Solo puedes radicar otro si es con una pagaduría diferente.", "request_id": "..."} }
```

Es la misma regla de capacidad que aplica Skala internamente; evita duplicar al cliente en la misma pagaduría.

6.2 GET /credits/:solicitud

Scope **credits:read**

Consultar crédito

Alternativas: GET /credits?solicitud=1420 o GET /credits?cedula=1020304050 (por cédula devuelve el más reciente de tu entidad). Solo se consultan créditos de tu entidad; cualquier otro responde 404.

Respuesta 200

```
{
  "id": "uuid",
  "solicitud_number": 1420,
  "estado": "EN ESTUDIO - ANALISTA",
  "monto": 5000000,
  "monto_desembolso": 4800000,
  "plazo": 60,
  "tasa": 1.85,
  "entidad": "CREDIALIANZA",
  "comision_estimada": 100000,
  "linea_credito": "LIBRE INVERSION",
  "cliente": { "nombre": "JUAN PEREZ", "documento": "1020304050", "correo": "juan@correo.com", "celular": "3001234567", "pagaduria": "Colpensiones" },
  "created_at": "2026-07-06T15:00:00Z",
  "updated_at": "2026-07-06T16:10:00Z"
}
```

6.2b GET /credits

Scope **credits:read**

Listar tus créditos

Sin `:solicitud` ni `?cedula=`, devuelve la lista de créditos de tu entidad (los más recientes primero).

Parámetro	Default	Notas
<code>limit</code>	20	Máximo 100 por página.
<code>offset</code>	0	Para paginar.
<code>estado</code>	—	Filtra por nombre exacto de estado (opcional).

Ejemplo: `GET /credits?limit=50&offset=0&estado=DESEMBOLSADO`

```
{
  "items": [ { "solicitud_number": 1420, "estado": "EN ESTUDIO - ANALISTA", "monto":
5000000, "entidad": "CREDIALIANZA", "cliente": { "nombre": "JUAN PEREZ", "documento":
"1020304050" }, "created_at": "..." } ],
  "limit": 50, "offset": 0, "count": 1
}
```

Página hasta que `count < limit`.

6.3 PATCH /credits/:solicitud/status

Scope `credits:update`

Cambiar estado

```
{ "estado": "DESEMBOLSADO", "motivo": "Desembolso confirmado" }
```

- `estado`: acepta el nombre completo del estado (ignora mayúsculas/acentos) o su `id`. Un nombre parcial se acepta solo si coincide con un único estado; si es ambiguo → 400 con `coincidencias`; si no se reconoce → 400 con `estados_validos`.
- `motivo`: opcional pero recomendado (queda en el historial y en un comentario de sistema).
- Si el estado es **DESEMBOLSADO**, se estampa automáticamente la fecha de desembolso.

Cada cambio registra una entrada en el **historial** y agrega un comentario de sistema (`[API] Estado → ...`).

Respuesta 200

```
{ "solicitud_number": 1420, "estado": "DESEMBOLSADO" }
```

BASH

```
curl -X PATCH
'https://yfosumpmtmcomfpbspaz.supabase.co/functions/v1/api/credits/1420/status' \
-H 'x-api-key: TU_API_KEY' -H 'Content-Type: application/json' \
-d '{"estado":"DESEMBOLSADO","motivo":"Confirmado"}'
```

6.4 PATCH /credits/:solicitud/status

Devolver crédito con tareas

Mismo endpoint, usando un **estado de devolución** (que habilita tareas) y el arreglo **tareas**.

Estados de devolución:

DEVUELTO

APLAZADO EN ESTUDIO

APROBADO PEND CERTIFICADOS

```
{
  "estado": "DEVUELTO",
  "motivo": "Faltan documentos del cliente",
  "tareas": [
    { "titulo": "Corregir la cédula (foto legible)", "requiereAdjunto": true },
    { "titulo": "Adjuntar desprendible actualizado", "requiereAdjunto": true },
    { "titulo": "Verificar la pagaduría" }
  ]
}
```

- Cada tarea puede ser un objeto { "titulo": string, "requiereAdjunto": boolean } o simplemente un **string** (título, sin adjunto obligatorio).
- Las tareas quedan como pendientes de subsanación; el gestor de Skala las verá y completará.
- Se registra en historial (**DEVOLUCIÓN CON TAREAS (API)**) y en un comentario de sistema con el detalle.

Respuesta 200

```
{ "solicitud_number": 1420, "estado": "DEVUELTO", "tareas": 3 }
```

BASH

```
curl -X PATCH 'https://yfosumpmtmcomfpbspaz.supabase.co/functions/v1/api/credits/1420/status' \
-H 'x-api-key: TU_API_KEY' -H 'Content-Type: application/json' \
-d '{"estado":"DEVUELTO","motivo":"Faltan docs","tareas":[{"titulo":"Corregir cédula","requiereAdjunto":true},"Verificar pagaduría"]}'
```

6.5 **POST** /credits/:solicitud/comments

Scope **credits:update**

Agregar comentario

```
{ "texto": "El cliente confirmó los datos de la cuenta bancaria.", "adjuntoUrl": "https://...", "adjuntoNombre": "soporte.pdf" }
```

- **texto**: obligatorio (máx 5000 caracteres).
- **adjuntoUrl** / **adjuntoNombre**: opcionales (URL pública de un soporte).

Respuesta **201**

```
{ "id": "uuid", "solicitud_number": 1420, "created_at": "2026-07-08T19:02:41Z" }
```

BASH

```
curl -X POST
'https://yfosumpmtmcomfpbspaz.supabase.co/functions/v1/api/credits/1420/comments' \
-H 'x-api-key: TU_API_KEY' -H 'Content-Type: application/json' \
-d '{"texto": "Cliente confirmó cuenta bancaria."}'
```

6.6 **GET** /credits/:solicitud/comments

Scope **credits:read**

Listar comentarios

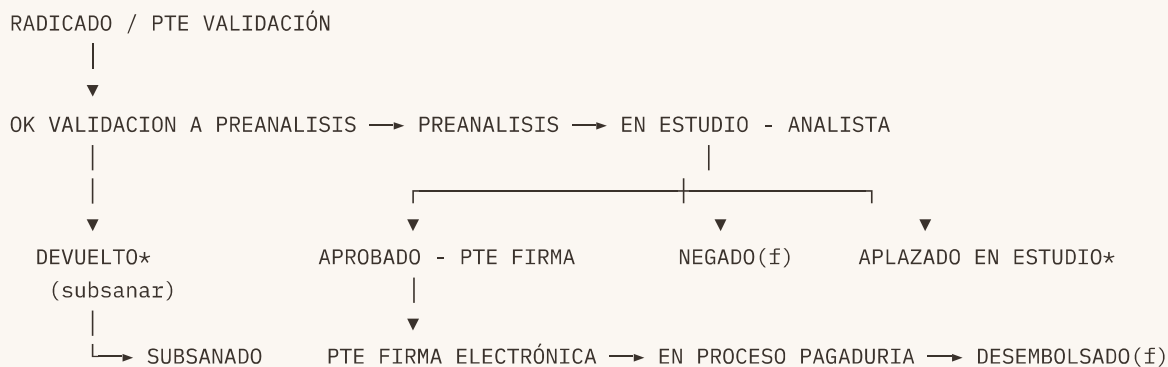
Respuesta **200**

```
{
  "solicitud_number": 1420,
  "comentarios": [
    { "texto": "Cliente confirmó cuenta bancaria.", "sistema": false, "adjunto": null,
      "adjunto_url": null, "fecha": "2026-07-08T19:02:41Z" },
    { "texto": "[API] Estado → DEVUELTO. Motivo: Faltan docs. Tareas: Corregir cédula (adjunto); Verificar pagaduría", "sistema": true, "adjunto": null, "adjunto_url": null,
      "fecha": "2026-07-08T19:04:36Z" }
  ]
}
```

Los comentarios vienen ordenados del más antiguo al más reciente. **sistema: true** = generado automáticamente por Skala.

7 Ciclo de vida del crédito

Flujo típico (los estados exactos y su orden los define Skala):



(f) = **estado final** (NEGADO, DESEMBOLSADO, DESISTIDO): el crédito no avanza más.

* = **estado de devolución** (habilita tareas de subsanación).

No hay validación de transiciones vía API: puedes mover el crédito a cualquier estado. **Respetar el flujo de negocio de Skala.**

8 Vía de vuelta · Skala → tu plataforma

Para enterarte cuando un crédito **tuyo** cambia de estado (p. ej. un analista de Skala lo aprueba o lo devuelve) hay **dos opciones**. Elige una:

OPCIÓN A

Polling

Recomendada si no tienes servidor de webhooks. Consultas el estado periódicamente; no expones ninguna URL.

- Sincroniza tu cartera: `GET /credits?limit=100&offset=0` cada X min y compara el estado.
- O consulta puntual: `GET /credits/1420`.
- Cadencia sugerida: cada 5–15 min. Simple, sin infraestructura extra.

OPCIÓN B

Webhooks firmados

Push en tiempo real, si tienes un endpoint HTTPS. Skala envía un `POST` JSON firmado a la URL que registres.

Solo recibes eventos de **tus** créditos.

Registro

Pásale a Skala tu **URL de webhook** (HTTPS). Skala la asocia a tu llave junto con un **secreto de firma** (`webhook_secret`) que te entregamos una vez. Con ese secreto verificas que el webhook vino de Skala.

Formato del evento — `credit_status_change`

```
{
  "event": "credit_status_change",
  "timestamp": "2026-07-14T17:24:48.435Z",
  "solicitud_number": 1420,
  "estado_anterior": "RADICADO / PTE VALIDACIÓN",
  "estado_nuevo": "EN ESTUDIO - ANALISTA",
  "credito": { "monto": 5000000, "monto_desembolso": 4800000, "plazo": 60, "tasa": 1.85,
  "entidad": "CREDIALIANZA", "comision_estimada": 100000, "linea_credito": "LIBRE INVERSION" },
  "cliente": { "nombre": "JUAN PEREZ", "documento": "1020304050", "correo": "juan@correo.com",
  "celular": "3001234567", "pagaduria": "Colpensiones" }
}
```

Headers: `X-Skala-Event: credit_status_change` y `X-Skala-Signature: sha256=<hmac>`.

Verificar la firma (obligatorio)

La firma es un **HMAC-SHA256** del **cuerpo crudo** (los bytes exactos que recibes) con tu `webhook_secret`. Compárala en tiempo constante:

JAVASCRIPT · NODE.JS

```
import crypto from 'crypto';
function verificar(rawBody, headerFirma, secret) {
  const esperado = 'sha256=' + crypto.createHmac('sha256', secret).update(rawBody,
'utf8').digest('hex');
  const a = Buffer.from(headerFirma || ''); const b = Buffer.from(esperado);
  return a.length === b.length && crypto.timingSafeEqual(a, b);
}
// if (!verificar(req.rawBody, req.header('X-Skala-Signature'),
process.env.SKALA_WEBHOOK_SECRET)) return res.sendStatus(401);
```

PYTHON

```
import hmac, hashlib
def verificar(raw: bytes, header: str, secret: str) -> bool:
  esperado = 'sha256=' + hmac.new(secret.encode(), raw, hashlib.sha256).hexdigest()
  return hmac.compare_digest(esperado, header or '')
```

 **Importante:** firma sobre el cuerpo crudo **sin re-serializar** (no vuelvas a `JSON.stringify`), o el HMAC no coincidirá.

Respuesta y reintentos

- Responde **2xx** rápido (idealmente < 5 s). Procesa de forma asíncrona.
- Si respondes distinto de 2xx o hay timeout, la entrega es **best-effort** (sin reintento automático en v1). Reconcilia con `GET /credits` periódicamente.
- **Idempotencia:** un mismo cambio podría llegar más de una vez; identifica por `solicitud_number + estado_nuevo + timestamp`.
- Si tú cambiaste el estado por la API, igual te llega el webhook (es el mismo evento); puedes ignorar los que correspondan a tu propia acción.

Aislamiento — solo recibes webhooks de **tus** créditos. Nunca eventos de otros aliados ni de la cartera interna de Skala.

Código	Significado
200 / 201	OK / creado
400	Falta un campo, valor fuera de rango, o estado no reconocido/ambiguo
401	API key ausente, inválida o vencida
403	La API key no tiene el scope requerido
404	Crédito o ruta no encontrada (incluye créditos fuera de tu alcance)
405	Método no soportado en esa ruta
409	Duplicado: ya hay un crédito en trámite para esa cédula/correo en la misma pagaduría (ver §6.1)
413	Body demasiado grande (máx 64 KB)
500	Error interno

Formato de error

```
{ "error": "mensaje legible", "request_id": "uuid-para-soporte" }
```

Skala **no expone** detalles internos; usa el `request_id` para reportar a soporte.

Casos especiales del `PATCH .../status`

- Estado ambiguo → { "error": "...", "coincidencias": ["...", "..."] }
- Estado no reconocido → { "error": "...", "estados_validos": ["...", "..."] }

Buenas prácticas

Idempotencia

Envía siempre **Idempotency-Key** al crear créditos, con un UUID por intento lógico. Evita duplicados en reintentos por timeout/red.

Reintentos

Ante 5x o timeout, reintenta con backoff exponencial y la **misma Idempotency-Key**. Ante 4xx, corrige la petición (no reintentes igual).

Estados

Usa el **nombre completo** o el **id** del estado para evitar ambigüedad; no uses abreviaturas.

Seguridad

Guarda la llave del lado servidor; una llave por entidad con scopes mínimos; róta la periódicamente.

Consulta antes de actualizar

Verifica el estado actual con **GET** si tu lógica depende de él (no hay bloqueo optimista).

11 Catálogos de referencia

Estados

Estado	Final	Habilita tareas
RADICADO / PTE VALIDACIÓN		
OK VALIDACION A PREANALISIS		
PREANALISIS		
EN ESTUDIO - ANALISTA		
PREAPROBADO		
APROBADO - PTE FIRMA		
APROBADO PEND CERTIFICADOS		✓
PTE FIRMA ELECTRÓNICA		
EN PROCESO PAGADURIA		
SUBSANADO		
DEVUELTO		✓
APLAZADO EN ESTUDIO		✓
NEGADO	✓	
DESISTIDO	✓	
DESEMBOLSADO	✓	

El nombre exacto es autoritativo — cópialo del **GET** o de esta tabla. Skala puede agregar estados; consulta el error `estados_validos` para la lista viva.

Entidades aliadas

CREDIALIANZA

COLTEFINANCIERA

VANTAGE

La Hipotecaria

Líneas de crédito

LIBRE INVERSION

COMPRA DE CARTERA

RETANQUEO

COMPRA + SANEAMIENTO

LIBRE + SANEAMIENTO

Pagadurías (texto libre; usa el nombre oficial)

Ejemplos frecuentes: Colpensiones · CASUR · CREMIL · Caja de Retiro de las Fuerzas Militares · Caja de Sueldos de Retiro de la Policía Nacional · Policía Nacional · Ejército Nacional · Pensionados Mindefensa · Fidupervisora · Consorcio FOPEP · FONCEP · Fondo Nacional de Prestaciones Sociales del Magisterio · Seguros de Vida Alfa S.A.

Tipos de documento

CEDULA (por defecto)

CÉDULA DE CIUDADANIA

CÉDULA DE EXTRANJERÍA

Preguntas frecuentes

¿Puedo crear el crédito sin gestor?

Sí; `gestorId` es opcional. Sin gestor, no se disparan notificaciones a gestor.

¿Cómo evito duplicar un crédito si mi request falla por timeout?

Usa `Idempotency-Key`; el reintento con la misma llave devuelve el crédito ya creado.

¿Puedo radicar el mismo cliente dos veces?

Sí, pero **no en la misma pagaduría** si el anterior sigue en trámite → responde **409**. Puedes con **otra pagaduría**, o cuando el anterior llegue a estado final. Para el mismo crédito lógico usa `external_ref/Idempotency-Key` para no duplicar por reintentos.

¿Cómo devuelvo un crédito para que corrijan algo?

`PATCH .../status` con un estado de devolución (ej. `DEVUELTO`) y el arreglo `tareas`.

¿La API valida que la transición de estado sea válida?

No; puedes mover a cualquier estado. Respeta el flujo de negocio.

¿Los cambios de estado disparan webhooks?

Sí — cualquier cambio de estado de tus créditos (por la API o por un analista de Skala) te llega como webhook firmado a tu URL registrada. Ver §8.

¿Cómo listo todos mis créditos?

`GET /credits?limit=50&offset=0` (paginado). Ver §6.2b.

¿Cómo obtengo la lista viva de estados?

Envía un **PATCH** con un estado inexistente y usa el arreglo **estados_validos** de la respuesta **400**.

CHANGELOG

- 1.2** 2026-07-15 · **Regla anti-duplicado por pagaduría (409)**: no se radica si ya hay un crédito en trámite para la misma cédula/correo en la misma pagaduría; respuesta **201** ahora incluye **external_ref**.
- 1.1** 2026-07-10 · Aislamiento por entidad; endpoint **GET /credits** (listar, paginado); campo **external_ref** (enlace de IDs + dedup); webhooks firmados (HMAC) + opción de polling; código **405** documentado.
- 1.0** 2026-07-08 · Versión inicial (crear, consultar, cambiar estado, devolver con tareas, comentar).



Soporte de integración — contacta a tu enlace comercial/técnico en Skala con el **request_id** de cualquier error.